

VinylEth — Final Project Report

Intermodular Project UD3 | CIFP Francesc de Borja Moll

Stepan Andreev

June 2026

Contents

1	Introduction	3
2	Objectives	3
2.1	Initial Objectives (set at UD1A)	3
2.2	Final State vs. Initial Goals	3
3	Technologies Used	4
3.1	Core Stack	4
3.2	Authentication and Security	4
3.3	Payments and Web3	5
3.4	External APIs	5
3.5	Email	5
3.6	Other	5
4	Architecture and Structure	6
4.1	Overview	6
4.2	Data Models	6
4.3	API Endpoints	7
4.4	State Management (Frontend)	8
5	Implemented Functionality	8
5.1	Catalog and Product Browsing	8
5.2	Shopping Cart	9
5.3	Wishlist	9
5.4	Checkout Flow	9
5.5	User Authentication	10
5.6	Admin Panel	10
5.7	Audio Player	11
5.8	PWA	11
6	Problems Encountered and Solutions	11
6.1	MongoDB crash on development machine	11
6.2	React fast-refresh lint error on CartContext	11
6.3	ETH-to-Wei conversion precision loss	12

6.4	React hook warning in wallet effect	12
6.5	Discogs API rate limiting in development	12
6.6	Anonymous orders and JWT auth boundary	12
7	Project Evolution	13
7.1	UD1A — Initial Setup	13
7.2	UD1B — Backend Integration and Cart	13
7.3	UD2A — Wallet UX, Search, and Error Contracts	13
7.4	UD3 — Full Application	14
8	Conclusions and Future Improvements	14
8.1	What Was Achieved	14
8.2	What Could Be Improved	15

1 Introduction

VinylEth is a full-stack web application for an online vinyl record store with integrated cryptocurrency payments. The project addresses a real market niche: vinyl has experienced a significant commercial revival over the last fifteen years, and a segment of its buyers are also participants in the Web3 ecosystem. VinylEth serves vinyl enthusiasts who value privacy and decentralised finance by letting them browse a curated catalog and complete purchases directly through MetaMask, without relying on traditional payment processors.

The application is built on the MERN stack (MongoDB, Express, React, Node.js) and integrates Ethereum's Sepolia testnet for payment transactions. The store side covers a full e-commerce flow: browsing, search and filtering, a persistent cart, a wishlist, user accounts, and checkout with receipt emails. The admin side provides a complete product management interface including direct Discogs API imports.

The project was developed over a full academic year across four phases: UD1A (initial setup), UD1B (backend integration and cart), UD2A (wallet UX, search, error contracts), and UD3 (payment flow, authentication, admin panel, email, and full polish).

2 Objectives

2.1 Initial Objectives (set at UD1A)

At the start of the project, the following goals were defined:

- Build a functional MERN e-commerce application as a foundation.
- Implement a product catalog connected to a real database.
- Build a persistent shopping cart.
- Integrate Ethereum payments via MetaMask as the distinguishing feature.
- Maintain professional-quality documentation throughout the course.

2.2 Final State vs. Initial Goals

All initial objectives were met and significantly exceeded. The final application includes areas that were not planned at the outset, such as a complete admin panel

with external API integrations, a wishlist system, an audio preview player, PWA support, and an order history module.

Objective	Planned	Delivered
Catalog from database	Yes	Yes
Shopping cart with persistence	Yes	Yes
MetaMask payment flow	Yes	Yes
User authentication	No	Yes — full JWT auth
Admin panel	No	Yes — full CRUD + stats
Email receipts	No	Yes — via Resend API
Audio previews	No	Yes — via iTunes API
Wishlist	No	Yes
Order history	No	Yes
PWA	No	Yes

3 Technologies Used

3.1 Core Stack

Technology	Version	Role
React	19	Frontend UI library (SPA)
React Router	7	Client-side routing
Vite / rolldown-vite	7	Frontend build tool
Node.js	20.19+	JavaScript runtime
Express	5	Backend HTTP framework
MongoDB	7	NoSQL database
Mongoose	9	MongoDB ODM

3.2 Authentication and Security

- **JWT (jsonwebtoken)** — stateless session tokens with 7-day expiry.
- **bcryptjs** — password hashing before storage.

- Custom middleware for both user auth (`requireAuth.js`) and admin access (`requireAdmin.js`) using separate token schemes.

3.3 Payments and Web3

- **MetaMask** — browser wallet used for transaction signing. No additional Web3 library was added; the native `window.ethereum` provider API (`eth_requestAccounts`, `eth_sendTransaction`) was used directly to keep dependencies minimal.
- All ETH amounts are converted to Wei using JavaScript `BigInt` to avoid floating-point precision errors.
- Payments target the Sepolia testnet.

3.4 External APIs

- **Discogs API** — fetches album metadata (title, artist, year, genre, label, country, barcode, tracklist with durations, and cover image) by release ID. Used in the admin panel's Quick Import feature.
- **iTunes Search API** — fetches a 30-second audio preview URL for a given artist and title. Used to auto-fill the audio preview field in the admin form.

3.5 Email

- **Resend** — transactional email service used to send order receipt emails after a successful MetaMask payment. On the free plan, the recipient is redirected to a configured fallback address.

3.6 Other

- **CORS** — enabled on the server for the configured `CLIENT_URL`.
 - **dotenv** — environment variable management.
 - **nodemon** — development auto-restart.
 - **PWA Manifest** — `public/manifest.json` with a custom SVG icon enables Add to Home Screen on mobile.
-

4 Architecture and Structure

4.1 Overview

The project follows a standard client-server separation. The frontend is a React SPA served by Vite during development. The backend is a REST API served by Express. They communicate over HTTP at localhost:5000 in development.

```
projecte-intermodular/
├─ client/                # React SPA
│  └─ src/
│     └─ pages/           # Full-page route components
│     └─ components/      # Reusable UI components
│     └─ api/             # Fetch wrapper functions
│     └─ services/        # Business logic (cart, price)
│     └─ context/         # React Contexts (Auth, Cart, Wishlist)
│     └─ config/          # App-level constants (store wallet)
│     └─ utils/           # Shared helpers
│  └─ public/             # Static assets, PWA manifest
└─ server/                # Express REST API
   └─ routes/             # Route definitions
   └─ controllers/        # Handler logic (albums)
   └─ models/             # Mongoose schemas
   └─ middleware/         # Auth and admin guards
   └─ utils/              # Helpers (Discogs mapper, email, price)
   └─ config/             # DB connection
   └─ seed/               # Initial dataset script
```

4.2 Data Models

Album

```
title, artist, year, genre, priceEth, coverUrl, stock,
featured (bool), description, audioUrl,
tracks: [{ title, duration }],
label, country, vinylFormat, barcode, mbid,
discountPercent
```

User

email (unique, lowercase), passwordHash,
savedAddress: { fullName, address, city, postalCode, country }

Order

txHash, totalEth,
items: [{ albumId, title, artist, qty, priceEth }],
shippingAddress: { fullName, address, city, postalCode, country },
userId (optional – anonymous checkout is supported),
createdAt

4.3 API Endpoints

Method	Endpoint	Auth	Description
GET	/api/albums	—	Paginated album list (search, filter, sort)
GET	/api/albums/genres	—	Distinct genre list
GET	/api/albums/:id	—	Single album detail
POST	/api/albums	Admin	Create album
PUT	/api/albums/:id	Admin	Update album
DELETE	/api/albums/:id	Admin	Delete album
POST	/api/auth/register	—	Register user
POST	/api/auth/login	—	Login, returns JWT
PUT	/api/auth/address	JWT	Save delivery address
POST	/api/auth/change-password	JWT	Change password
GET	/api/orders/mine	JWT	User order history
POST	/api/orders	—	Create order after payment
GET	/api/admin/stats	Admin	Dashboard stats
GET	/api/admin/orders	Admin	All orders
POST	/api/admin/login	—	Admin login

Method	Endpoint	Auth	Description
GET	/api/discogs/:id	Admin	Fetch release from Discogs
GET	/api/itunes	Admin	Fetch audio preview from iTunes

4.4 State Management (Frontend)

React Context is used for all global state. Three contexts exist:

- **AuthContext** — current user, JWT token, login/logout/register actions. Token is persisted to localStorage.
- **CartContext** — cart items, quantities, totals, add/increment/decrement/remove/clear. Cart is persisted to localStorage.
- **WishlistContext** — wishlist items, toggle. Persisted to localStorage.

No third-party state management library was used; the combination of Context and custom hooks (useAuth, useCart, useWishlist) is sufficient for the application’s scale.

5 Implemented Functionality

5.1 Catalog and Product Browsing

The catalog page loads albums from the backend API with server-side pagination (“Load more”). Users can search across title, artist, and genre using a debounced input (250 ms delay to avoid excessive requests). A genre dropdown fetches available genres from the database at runtime. Albums can be sorted by title, price, year, stock quantity, and featured status.

Each album card shows the cover, title, artist, price in ETH, a stock status badge (Out of Stock / Last Copies at stock ≤ 2), and a wishlist toggle. Clicking through opens a detail page with full metadata, a tracklist with individual track durations, a 30-second audio preview player (when an iTunes URL is available), a similar albums section, and an add-to-cart button.

The homepage shows a featured album section and a special offers section (albums with a non-zero discount). A recently viewed shelf tracks the last 6 albums opened, stored in `localStorage`. Skeleton loading cards are shown while data is fetching.

5.2 Shopping Cart

Cart state is managed globally through `CartContext`. Items can be added from both the catalog card and the detail page. The cart enforces stock limits: quantity cannot exceed the available stock, and adding an out-of-stock album is blocked. The quantity can be incremented or decremented per line; decrement at 1 removes the item. Prices respect any active discount via a shared `effectivePrice()` utility. The full cart state persists across page refreshes through `localStorage`.

5.3 Wishlist

Any album can be toggled into or out of the wishlist from the catalog or detail page. A badge in the header shows the wishlist count. The wishlist page renders the saved albums as cards with the same add-to-cart action. State persists to `localStorage`.

5.4 Checkout Flow

Checkout is a three-step linear flow: Cart → Shipping → Payment.

The shipping step presents a validated form (all fields required). If the logged-in user has a saved address on their profile, it is pre-filled automatically. Submitting the shipping form saves the address to the user's profile via the API and navigates to the payment step.

The payment step shows a summary of the order (items, quantities, total in ETH) and a "Pay with MetaMask" button. On click:

1. The app requests a chain switch to Sepolia (chain ID `0xaa36a7`) if needed.
2. `eth_sendTransaction` is called with the `STORE_WALLET` address and the total in Wei (converted via `BigInt` to avoid float imprecision).
3. On success, the transaction hash is displayed with a direct Etherscan link.
4. A `POST /api/orders` request saves the order (items, shipping, tx hash, total) to the database.
5. A receipt email is sent via Resend.

6. The cart is cleared.

5.5 User Authentication

Users register with an email and password (minimum 6 characters, duplicate email rejected). Passwords are hashed with bcrypt before storage. Login returns a JWT with a 7-day expiry, stored in `localStorage` and attached as a Bearer token on authenticated requests. The `Router.jsx` protects routes that require login; unauthenticated users are redirected to `/login`. The profile page lets users view their email, change their password, and manage their saved delivery address. An order history page shows all past orders with TX hashes, Etherscan links, items, and totals.

5.6 Admin Panel

The admin panel is accessed at `/admin` with a separate password-only login (not connected to user accounts). The session token is stored under a different `localStorage` key and sent as an `x-admin-token` header.

The dashboard shows live stats: total album count, out-of-stock albums, total orders, and total revenue in ETH, plus a top 5 best-selling albums list.

The album management tab supports:

- Creating and editing albums through a full form.
- Auto-filling the form from Discogs by entering a release ID (title, artist, year, genre, label, country, barcode, tracklist, cover URL).
- Auto-filling the audio preview URL from iTunes.
- Deleting individual albums.
- Batch operations: select multiple albums and set a discount percentage, toggle featured status, or delete.
- Exporting the current filtered album list to CSV.

The orders tab shows all orders with buyer email (or “Anonymous”), items, total ETH, TX hash, and date. The list can be filtered by a date range or searched by TX hash substring.

5.7 Audio Player

Albums that have an `audioUrl` field (a 30-second iTunes preview) display a custom HTML5 audio player on their detail page. The player includes play/pause, a seekable progress bar with current time and total duration display, and a buffering indicator.

5.8 PWA

A `manifest.json` in the client's public directory defines the app name, theme colors, display mode (standalone), and a custom SVG icon. This enables Add to Home Screen prompts on supported mobile browsers.

6 Problems Encountered and Solutions

6.1 MongoDB crash on development machine

During the UD2A session, the system's MongoDB package (`mongodb-bin 8.2.7` on an Arch-based Linux) crashed with a SIGSEGV signal at startup, making the backend unable to connect.

Solution: Switched to a containerised MongoDB 7 instance via Docker (`docker run -d --name mongo -p 27017:27017 mongo:7`). This provided a stable database for the rest of development without changing any application code, since the connection string `mongodb://127.0.0.1:27017/vinyleth` still resolved correctly.

6.2 React fast-refresh lint error on CartContext

After splitting cart logic into `CartContext.jsx`, the ESLint React fast-refresh plugin rejected the file because it mixed a context provider export with a hook export.

Solution: Extracted the hook (`useCart.js`) and the context value builder (`cartContextValue.js`) into dedicated files. The provider file kept only the provider component, which satisfied the lint rule.

6.3 ETH-to-Wei conversion precision loss

An early implementation of the payment conversion used JavaScript floating-point arithmetic (`amount * 1e18`), which produced rounding errors for prices like 0.015 ETH due to IEEE 754 float representation.

Solution: Replaced the calculation with `BigInt` arithmetic. The ETH amount is converted to a string, split at the decimal point, and each part is scaled independently before summing, eliminating any float imprecision.

6.4 React hook warning in wallet effect

The initial MetaMask wallet detection ran inside a `useEffect`, which set state synchronously within the effect body. This triggered a `set-state-in-effect` lint warning.

Solution: Moved the initial wallet availability check to the `useState` initializer function so it runs once at component creation rather than inside an effect.

6.5 Discogs API rate limiting in development

During album seeding and testing of the Quick Import feature, repeated requests to the Discogs API without a token resulted in 429 Too Many Requests responses.

Solution: Added a `DISCOGS_TOKEN` environment variable. The server attaches it as an `Authorization: Discogs token=...` header on all Discogs requests, which raises the rate limit to 60 requests per minute.

6.6 Anonymous orders and JWT auth boundary

The checkout flow allows both logged-in and anonymous users to complete a purchase (since requiring an account before buying creates unnecessary friction). However, the `POST /api/orders` endpoint needed to optionally attach a `userId` without failing when no token was present.

Solution: Created a lenient middleware that attempts to decode the JWT if a Bearer token is present, attaches `req.user` if successful, and calls `next()` regardless. The order route then sets `userId: req.user?._id ?? null`, making the field optional at the schema level as well.

7 Project Evolution

7.1 UD1A — Initial Setup

The project started with a basic React frontend and an Express server skeleton. The frontend displayed a static album catalog using hardcoded mock data. There was no database and no routing. The design identity (retro/vintage aesthetic, cream and burnt orange palette, Playfair Display typography) was established at this stage.

Key deliverables: project structure, working Express server, React app with mock catalog, initial README, design system.

7.2 UD1B — Backend Integration and Cart

UD1B focused on replacing mocks with real backend data. MongoDB was connected and an Album model was defined. Two API endpoints were added (GET /api/albums and GET /api/albums/:id). The frontend was refactored to fetch data from the API, eliminating all mock data. An album detail page was added with client-side routing. A cart was built using React Context with localStorage persistence.

The folder structure was reorganised into pages/, components/, api/, and services/ to establish a clear separation of concerns that would scale through the rest of the project.

Key deliverables: MongoDB connection, album API, frontend API integration, album detail page, cart with persistence, folder restructure.

7.3 UD2A — Wallet UX, Search, and Error Contracts

UD2A added the first visible Web3 integration: a MetaMask wallet connect/disconnect flow in the header, showing the user's shortened address once connected. Cart behavior was deepened with increment/decrement controls, stock-capped quantities, and per-line subtotals. The catalog gained real-time search, genre filtering, and sorting.

API error handling was hardened: the backend now distinguishes between malformed IDs (400) and valid-but-missing IDs (404), and the frontend surfaces the actual error message from the response body rather than a generic string.

Two test scripts were added: a backend API contract checker and a frontend cart state transition checker.

Key deliverables: MetaMask connect, cart quantity controls, catalog search/filter/sort, API error contracts, test scripts.

7.4 UD3 — Full Application

UD3 completed the remaining backlog items and raised the project to production-ready quality. The major additions were:

- **Payment flow:** full `eth_sendTransaction` integration on the Sepolia testnet, with ETH→Wei conversion, TX hash display, and Etherscan links.
- **Authentication:** user register/login with JWT, protected routes, profile page, password change, saved delivery address.
- **Orders:** order creation on payment success, stock decrementation, user order history page.
- **Email:** Resend integration to send order receipts after payment.
- **Admin panel:** full product management CRUD, batch operations, Discogs Quick Import, iTunes audio fill, CSV export, order management with search and date filter, dashboard stats.
- **Wishlist:** toggle, dedicated page, count badge, `localStorage` persistence.
- **Enriched catalog:** featured album section, special offers, similar albums, recently viewed, skeleton loading cards, stock badges.
- **Audio player:** custom HTML5 player with seek bar and buffering indicator.
- **PWA:** manifest and icon for mobile Add to Home Screen.

8 Conclusions and Future Improvements

8.1 What Was Achieved

VinylEth grew from a static mock catalog into a complete, multi-role web application over four phases. It covers the full customer journey from browsing to crypto payment, includes a production-grade admin panel, and integrates three external APIs. The codebase is organized clearly enough to be understood and extended by a new

developer.

The project demonstrates several technically non-trivial integrations: stateless JWT auth, BigInt-safe ETH payment conversion, Discogs metadata mapping, iTunes audio previews, and transactional email — all working together in a single application.

8.2 What Could Be Improved

Smart contract instead of direct transfer. The current payment sends ETH directly to a wallet address. A real deployment would use a Solidity escrow contract that holds funds until the order is fulfilled, providing buyer protection and enabling refunds.

Real stock management. Stock is decremented at order creation, but there is no reservation mechanism during checkout. Two users completing checkout simultaneously could both succeed even if only one unit is available.

Image upload. Album cover images are currently entered as URLs (including from Discogs). An upload feature with server-side storage (e.g. Cloudinary) would be more practical for store management.

Automated test suite. The existing test scripts are manual contract checks. A proper test suite with Vitest for the frontend and Jest + Supertest for the backend would prevent regressions as the application grows.

Pagination improvements. The current “Load more” approach loads additional items into the same list. Server-side cursor-based pagination would be more efficient at scale.

Admin role as user flag. The current admin authentication is a separate password-based system independent of user accounts. Integrating admin access as a role flag on the User model would unify the auth system and allow per-account admin grants.

VinylEth — Stepan Andreev — Projecte Intermodular UD3 — CIFP Francesc de Borja Moll — 2025-2026